INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2026

COURSE: [S26] DevOps and Security

LS Lab 1 REPORT

On

Containerization and application layer load balancing

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 26.01.2026

TASK 1: Get familiar with Docker Engine & NGINX

1. Pull *Nginx v1.23.3* image from *dockerhub* registry and confirm it is listed in local images

Solution:

Checking the presence of Docker:

The following command was used to verify whether Docker was installed on the system and the current version of Docker.

```
hasib@hasib-VMware-Virtual-Platform:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1
```

The output showed Docker version 28.2.2, which indicates that Docker was properly installed and usable.

Installing Docker at Linux Machine:

The package list has been updated with the following command.

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt update
Hit:1 http://ru.archive.ubuntu.com/ubuntu questing InRelease
Hit:2 http://ru.archive.ubuntu.com/ubuntu questing-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu questing-security InRelease
Hit:4 http://ru.archive.ubuntu.com/ubuntu questing-backports InRelease
74 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Docker and Docker Compose have been installed with the following command.

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt install -y docker.io docker-compose
Note, selecting 'docker-compose-v2' instead of 'docker-compose'
docker.io is already the newest version (28.2.2-0ubuntu1).
docker-compose-v2 is already the newest version (2.37.1+ds1-0ubuntu2).
The following packages were automatically installed and are no longer required:
  linux-headers-6.17.0-5          linux-tools-6.17.0-5
  linux-headers-6.17.0-5-generic  linux-tools-6.17.0-5-generic
  linux-modules-6.17.0-5-generic
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 74
```

The output showed that Docker and Docker Compose were already installed and were the latest versions. No new packages were installed.

Starting the Docker service:

The following command starts the Docker service and sets Docker to start automatically when the system boots.

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo systemctl enable --now docker
```

Adding a user to the docker group:

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo usermod -aG docker $USER
```

By running the above command, the Docker service has been successfully started and the current user has been added to the Docker group.

After logging out and logging in, the following command has been executed:

The following command shows the currently running Docker containers. As a result, an empty table is shown, which means that Docker is running properly but there are no containers running yet.

```
hasib@hasib-VMware-Virtual-Platform:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

2. Run the pulled *Nginx* as a container with the below properties

a. Map the port to 8080.

b. Name the container as *nginx-<stX>*.

c. Run it as daemon.

Solution:

NGINX Image Pull (version 1.23.3):

Executing the following command will download the NGINX image of the specified version 1.23.3 from the *DockerHub* registry to the local system.

```
hasib@hasib-VMware-Virtual-Platform:~$ docker pull nginx:1.23.3
1.23.3: Pulling from library/nginx
f1f26f570256: Pull complete
84181e80d10e: Pull complete
1ff0f94a8007: Pull complete
d776269cad10: Pull complete
e9427fcfa864: Pull complete
d4ceccbfc269: Pull complete
Digest: sha256:f4e3b6489888647ce1834b601c6c06b9f8c03dee6e097e13ed3e28c01ea3ac8c
Status: Downloaded newer image for nginx:1.23.3
docker.io/library/nginx:1.23.3
```

This results in multiple layers Pull complete, which means that all the required parts of the image have been successfully downloaded. Status: Downloaded newer image is displayed, which confirms that the image has been successfully pulled.

Viewing the local image list:

The following command shows a list of all the Docker images on the local machine. The output shows the nginx repository and the 1.23.3 tag, which proves that the image has been successfully saved to the local system.

```
hasib@hasib-VMware-Virtual-Platform:~$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        1.23.3    ac232364af84   2 years ago   142MB
```

Running default NGINX Container:

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker run -d \
--name nginx-st24 \
-p 8080:80 \
nginx:st24
2d73094d061ba5f933243a2a85e341886ae1f9cf2ea11f0c3d930322603242ac
```

After creating and starting a new container with docker run, the container is run in daemon (background) mode with *-d*. The *--name* nginx-st24 specifies the name of the container. The *-p 8080:80* maps the host's port *8080* to the container's *port 80. nginx:1.23.3* is the image and version used. The output shows a long container ID, indicating that the container was successfully created and started.

Checking if the container is running:

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker ps
CONTAINER ID   IMAGE        COMMAND                  CREATED          STATUS          PORTS                                       NAMES
2d73094d061b   nginx:st24   "/docker-entrypoint.…"   11 seconds ago   Up 10 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   nginx-st24
```

The output shows:

- Container name: *nginx-st24*
- Image: nginx:*1.23.3*
- Port mapping: *8080 -> 80*

This indicates that the NGINX container is running successfully and the port mapping is working properly.

3. Confirm port mapping.

a. List open ports in host machine.

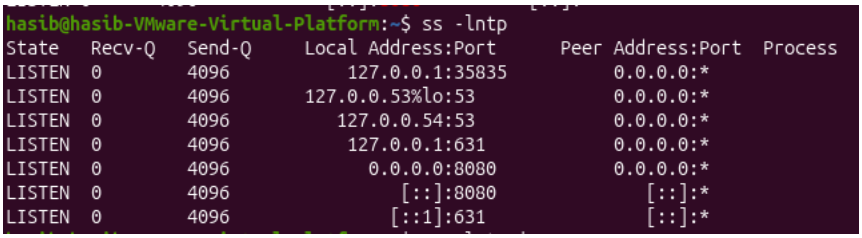
b. List open ports inside the running container.

c. Access the page from your browser.

Solution:

Port checking on host machine:

To verify the port mapping, first I have checked the open ports on the Host machine. For this purpose, the following command has been used to see which TCP ports were currently listening. The output showed that both *0.0.0.0:8080* and *[::]:8080* were in the LISTEN state, so port *8080* on the Host machine was active for both IPv4 and IPv6.

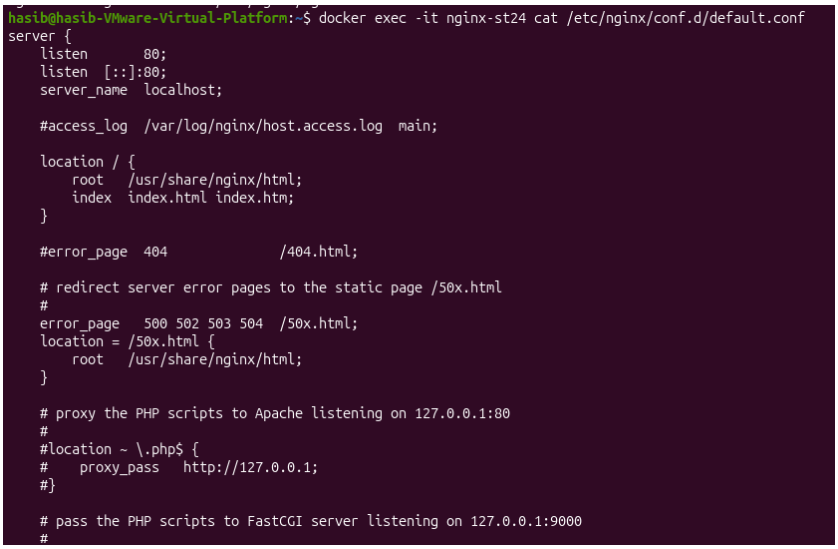


The following command was run to separately verify only the information for port *8080*, which confirmed that port *8080* is properly open and listening on the host side.



List of open ports inside the running container:

The default configuration file of NGINX was viewed using the following command. The lines listen *80*; and listen *[::]:80*; in the configuration file confirmed that the NGINX web server inside the container was listening on port *80* on both IPv4 and IPv6 addresses. This verified the container-side internal port.

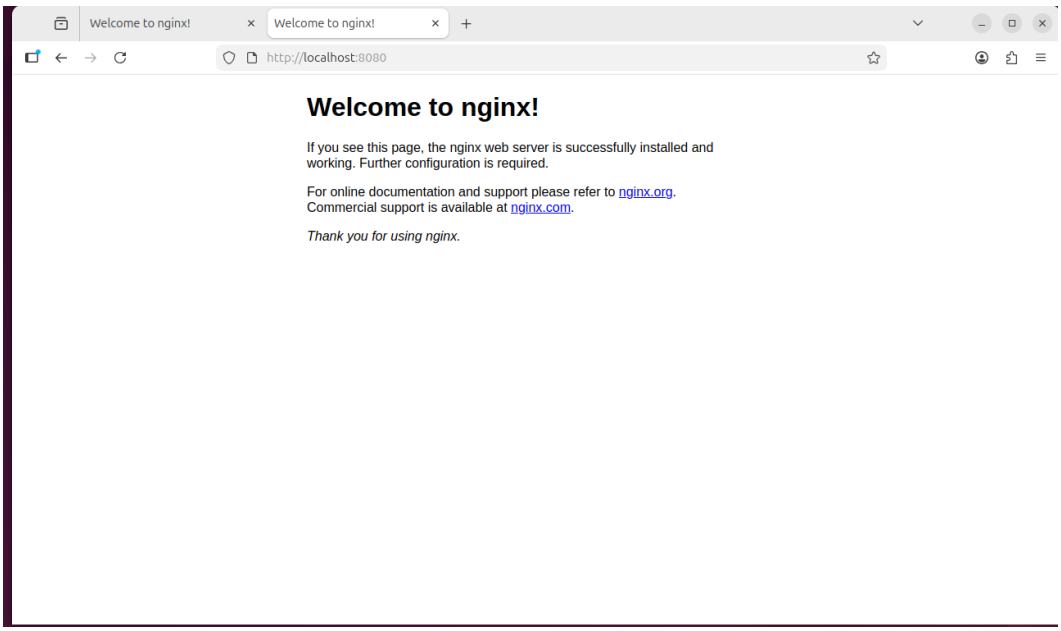


```
    #location ~ \.php$ {
    #    root           html;
    #    fastcgi_pass   127.0.0.1:9000;
    #    fastcgi_index  index.php;
    #    fastcgi_param  SCRIPT_FILENAME  /scripts$fastcgi_script_name;
    #    include        fastcgi_params;
    #}

    # deny access to .htaccess files, if Apache's document root
    # concurs with nginx's one
    #
    #location ~ /\.ht {
    #    deny  all;
    #}
}
```

Checking browser:

I opened Firefox and entered the link *http://localhost:8080*. Successfully Default Welcome to *nginx!* was arrived.



4. Create a Dockerfile similar to the below properties (let's call it container A).

a. Image tag should be *Nginx v1.23.3.*

b. Create a custom *index.html* file and copy it to your docker image to replace the Nginx default web page.

c. Build the image from the *Dockerfile*, tag it during build as *nginx:<stX>*, check/validate local images, and run your custom made docker image.

d. Access via browser and validate that your custom page is hosted.

Solution:

In the *Dockerfile* creation step, first create a Working Directory:

Here a separate working folder has been created in which the Dockerfile and index.html are in the same place.

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ mkdir -p ~/lab2-st24/task2
```

After the folder was created, I went to that directory.

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ cd ~/lab2-st24/task2
```

Creating LARGE Human-Made index.html:

A new HTML file was created/edited with the following command. This file later replaced the default welcome page of NGINX.

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ nano Dockerfile
```

This is a large and professionally designed static web page, which clearly displays information such as Student ID, Docker, NGINX version, port mapping, etc.

```
 GNU nano 8.4                                                    index.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Custom LS Lab 1 index.html - Student 24</title>

    <style>
        body {
            margin: 0;
            font-family: "Segoe UI", Arial, sans-serif;
            background: linear-gradient(to right, #0f2027, #203a43, #2c5364);
            color: #ffffff;
        }

        header {
            background-color: #111;
            padding: 25px;
            text-align: center;
            border-bottom: 4px solid #00c6ff;
        }

        header h1 {
            margin: 0;
            font-size: 36px;
            letter-spacing: 2px;
        }

        header p {
            margin-top: 10px;
            font-size: 18px;
            color: #cccccc;
        }

        section {
            padding: 50px 10%;
```

```
GNU nano 8.4                                                    index.html
            padding: 50px 10%;
        }

        .card {
            background-color: rgba(255,255,255,0.08);
            padding: 30px;
            border-radius: 12px;
            margin-bottom: 30px;
            box-shadow: 0 8px 20px rgba(0,0,0,0.3);
        }

        .card h2 {
            color: #00e6ff;
            margin-bottom: 15px;
        }

        .card p {
            line-height: 1.8;
            font-size: 17px;
        }

        .info-grid {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
            gap: 20px;
            margin-top: 30px;
        }

        .info-box {
            background-color: rgba(0,0,0,0.35);
            padding: 20px;
            border-radius: 10px;
            border-left: 5px solid #00c6ff;
        }
```

```
GNU nano 8.4                                                    index.html
        footer {
            background-color: #111;
            padding: 20px;
            text-align: center;
            font-size: 14px;
            color: #aaaaaa;
        }

        footer span {
            color: #00e6ff;
        }
    </style>
</head>

<body>

<header>
    <h1>Docker + NGINX Web Server</h1>
    <p>Task 1 | Student ID: 24 | Container: nginx-st24</p>
</header>

<section>

    <div class="card">
        <h2>About This Project</h2>
        <p>
            This website is served from a Docker container running the
            <strong>NGINX 1.23.3</strong> image. The container is deployed using Docker Engine
            on an Ubuntu machine.
        </p>
    </div>

    <div class="card">
        <h2>Technical Information</h2>

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy
```

```
GNU nano 8.4                                                    index.html
    <div class="card">
        <h2>Technical Information</h2>

        <div class="info-grid">
            <div class="info-box">
                <strong>Operating System</strong><br>
                Ubuntu
            </div>

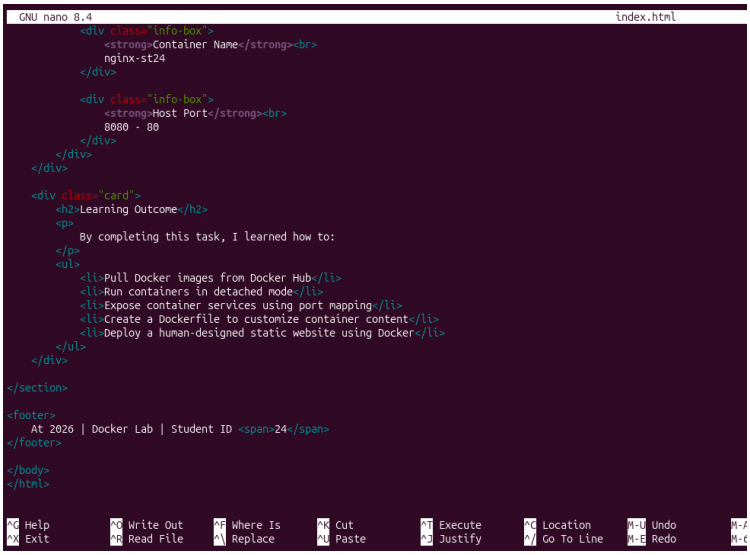
            <div class="info-box">
                <strong>Web Server</strong><br>
                NGINX 1.23.3
            </div>

            <div class="info-box">
                <strong>Container Name</strong><br>
                nginx-st24
            </div>

            <div class="info-box">
                <strong>Host Port</strong><br>
                8080 - 80
            </div>
        </div>
    </div>

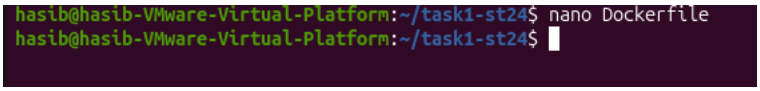
    <div class="card">
        <h2>Learning Outcome</h2>
        <p>
            By completing this task, I learned how to:
        </p>
        <ul>
            <li>Pull Docker images from Docker Hub</li>
            <li>Run containers in detached mode</li>

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo
```



Creating a *Dockerfile*:

A new *Dockerfile* has created using the following command. This *Dockerfile* has specified which base image to use and how to copy the custom index.html file to the default web directory of NGINX.



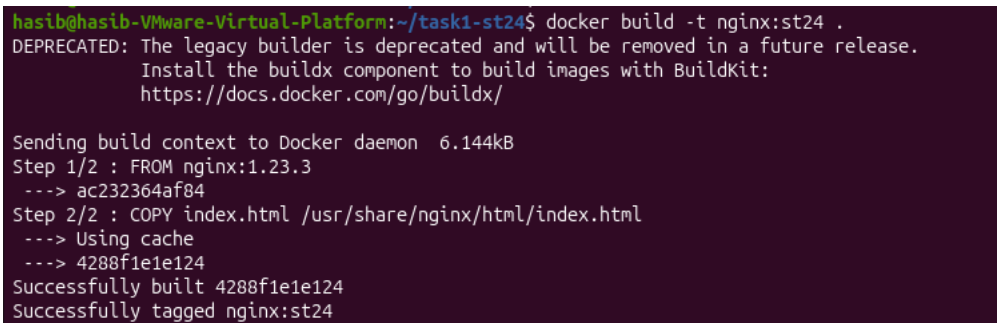
The following content has been input inside it:



FROM nginx:1.23.3 means that the image will be based on NGINX version 1.23.3. COPY index.html /usr/share/nginx/html/index.html copies the custom index.html file to the default web root of NGINX inside the container, replacing the default welcome page.

Building Custom Docker Image:

This command has built a new Docker image according to the *Dockerfile* and has set the image's tag to nginx:st24 during the build.



This command has pulled the base image into a Docker build process as per the *Dockerfile*, then has copied *index.html* and finally showed Successfully tagged *nginx:st24* to indicate that the image was successfully built.

Verify Local Image:

The following command has showed all the Docker images on the local system and only the custom image has filtered using *grep st24.*

A new Docker image has been built and *nginx:st24* has been set as the tag of the image during the build.

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker images | grep st24
nginx        st24      4288f1e1e124   11 hours ago   142MB
```

nginx st24 showed to confirm that the custom image exists on the local system.

Stop and Remove Old Container:

Before running a new container with the same name, the old *nginx-st24* container has been first stopped and removed.

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker stop nginx-st24 2>/dev/null || true
nginx-st24
```

Removing:

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker rm nginx-st24 2>/dev/null || true
nginx-st24
```

After the container has been stopped and removed, its name or ID has been displayed. This indicated that the old container has been no longer running.

Custom Container Run:

This command has started a new container using the custom image *nginx:st24*, where the *-d* option enables daemon/background mode. *--name nginx-st24* was the name of the container, and *-p 8080:80* mapped host port *8080* to container port *80*.

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker run -d \
--name nginx-st24 \
-p 8080:80 \
nginx:st24
2d73094d061ba5f933243a2a85e341886ae1f9cf2ea11f0c3d930322603242ac
```

A long container ID has been found in the output, which indicates that the container ran successfully.

Checking:

The status of currently running containers has been checked using the command.

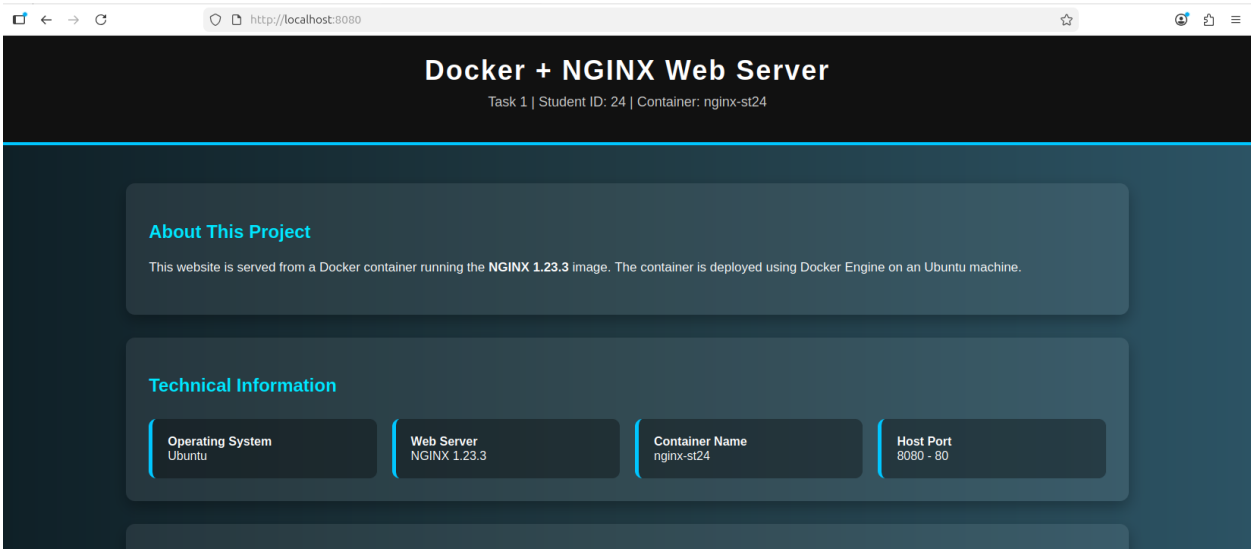
```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker ps
CONTAINER ID   IMAGE        COMMAND                  CREATED          STATUS          PORTS                                       NAMES
2d73094d061b   nginx:st24   "/docker-entrypoint.…"   11 seconds ago   Up 10 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   nginx-st24
```

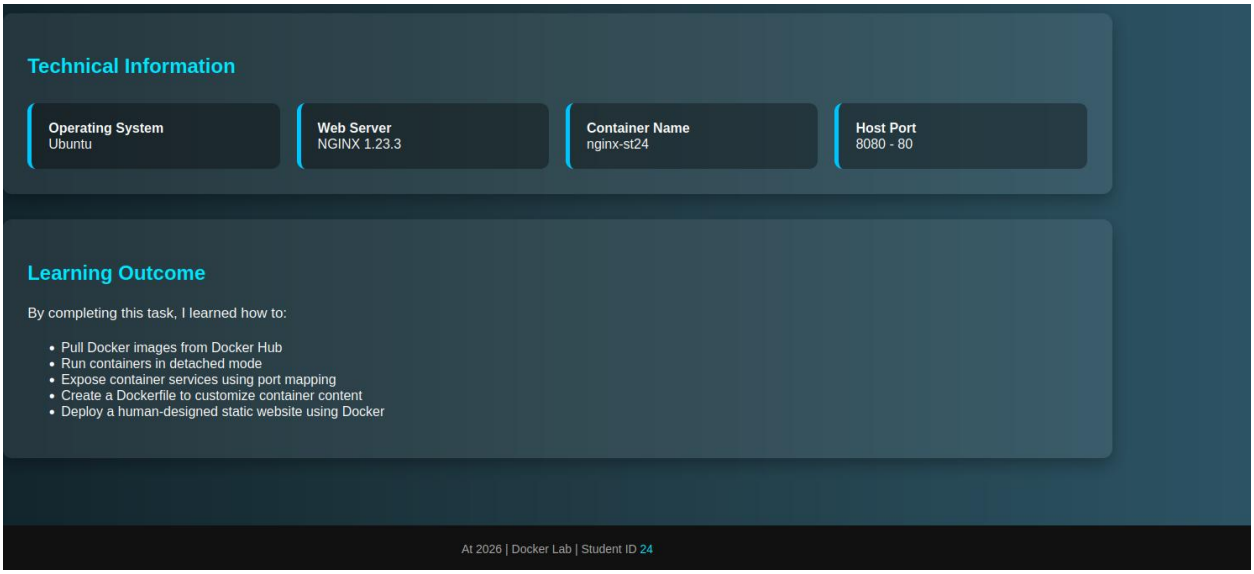
In the output, the nginx-st24 container is shown as being in an Up status, indicating that the container is running and the port mapping is working properly.

Final Verification:

Opening the browser and entering the following URL:

http://localhost:8080





Instead of showing the default “Welcome to nginx!” in the browser, the newly created website clearly displays information about Student 24, Docker, NGINX, and Task-1.

TASK 2: Work with multi-container environment

1. Create another Dockerfile similar to step 1.4 (Let’s call it container B), and an *index.html* with different content.

Solution:

The previous *nginx-st24* container was running, so using the same port/name may cause a conflict. So first, the container has been stopped and removed. Using *2>/dev/null || true* will not show an error even if the container is not there, and the command will continue.

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker stop nginx-st24 2>/dev/null || true
nginx-st24
```

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker rm nginx-st24 2>/dev/null || true
nginx-st24
```

Also, because Docker Compose was run earlier, it has been taken down.

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ docker compose down 2>/dev/null || true
```

The output shows *nginx-st24* which means the container has been successfully stopped and removed. Also, compose running has stopped it.

Container B creation:

A separate workspace folder has been created and entered there. Then, preparation has been made to keep the *containerA/, containerB/, loadbalancer/* type structure here.

Project folder creation:

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ mkdir -p ~/lab2-st24/task2
```

```
hasib@hasib-VMware-Virtual-Platform:~/task1-st24$ cd ~/lab2-st24/task2
```

Then 3 folders were created:

- *containerA/*
- *containerB/*
- *loadbalancer/*

Container A folder + *Dockerfile* + index.html:

Container A has been created as a reference, so that Container B has followed the "A-like" *Dockerfile* structure. Here, a *containerA* folder has been created, and an *index.html* and *Dockerfile* have been placed inside it.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ mkdir -p containerA
```

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ cd containerA
```

Then *index.html* has been created:

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2/containerA$ nano index.html
```

```
 GNU nano 8.4                                                    index.html
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>Container A - st24</title>
  <style>
    body{font-family:Arial;margin:0;background:#0b1220;color:#e8eefc}
    header{padding:25px;background:#111a2c;border-bottom:4px solid #4aa3ff}
    h1{margin:0;font-size:34px}
    .wrap{padding:35px}
    .card{background:rgba(255,255,255,.06);padding:22px;border-radius:12px;box-shadow:0 10px 25px rgba(0,0,0,.35)}
    .tag{display:inline-block;margin-top:12px;padding:8px 12px;border-radius:999px;background:#4aa3ff;color:#001428;font-weight:bold}
  </style>
</head>
<body>
<header>
  <h1>Container A (Student ID 24)</h1>
  <div class="tag">Port 8080 - Nginx</div>
</header>
<div class="wrap">
  <div class="card">
    <h2>Service A is running</h2>
    <p>This page is hosted by Container A built via docker-compose (multi-build).</p>
  </div>
</div>
</body>
</html>

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo    M-A Set Mark   M-] To Bracket  M-B Previou
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo    M-6 Copy       ^B Where Was    M-F Next
```

A Dockerfile has been created:

```
 GNU nano 8.4                                                    Dockerfile
FROM nginx:1.23.3
COPY index.html /usr/share/nginx/html/index.html
```

Then returned to root:

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2/containerA$ cd ..
```

The files were created with the Nano editor open. The presence of *FROM nginx:1.23.3* and *COPY …* in the *Dockerfile* indicates that A's custom page has replaced the default NGINX page using the *NGINX 1.23.3* base.

Different container B:

Here, a separate folder has been created for Container B, with a separate *index.html* and a *Dockerfile* of the same pattern as A.

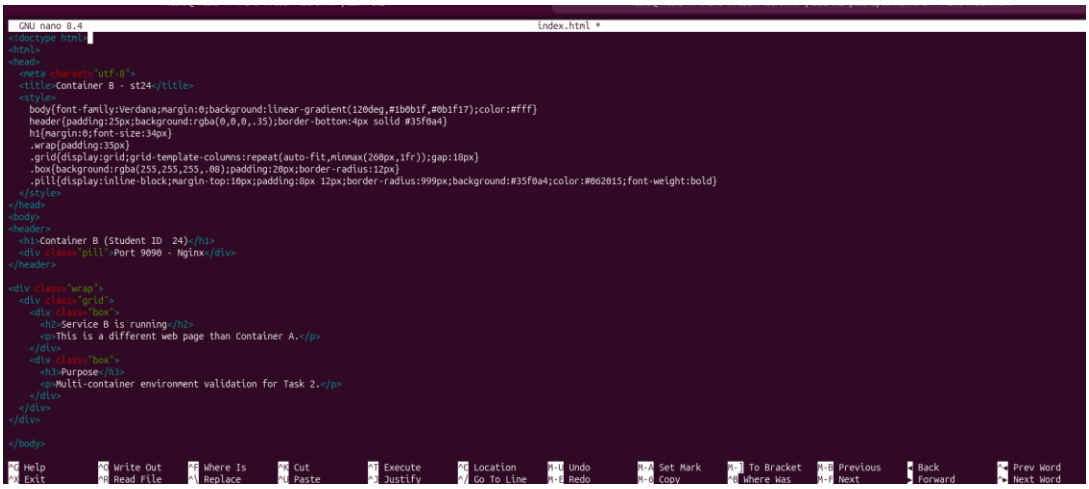
```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ mkdir -p containerB
```

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ cd containerB
```

Then B's index.html is created:

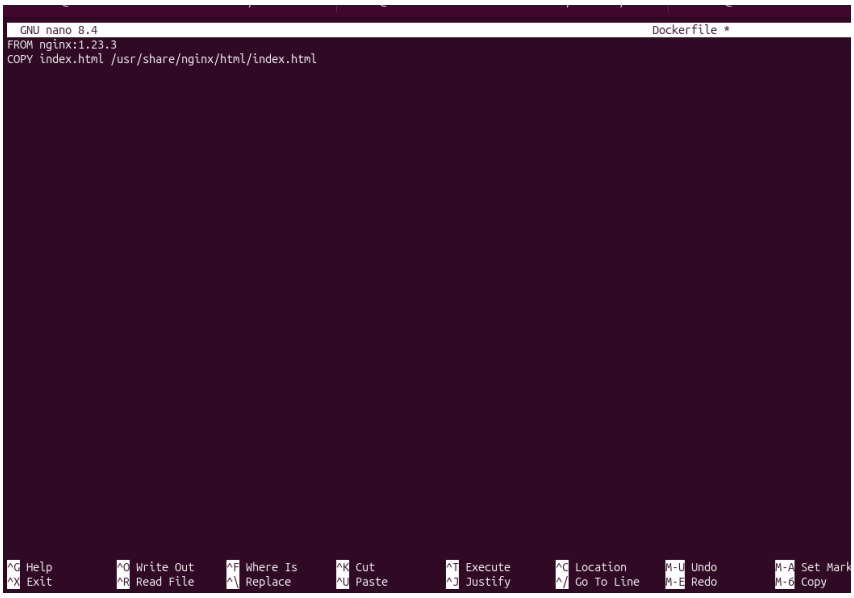
```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2/containerB$ nano index.html
```

This HTML file has a different design, text, and heading from A.



Then B's Dockerfile is created:

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2/containerB$ nano Dockerfile
```



Then returned to root:

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2/containerB$ cd ..
```

2. Write a docker-compose file with the below properties

a. Multi-build: Builds both Dockerfiles and runs both images.

b. Port mapping: Container A should listen to port 8080 and container B should listen to port 9090. (They host two different web pages)

c. Confirm both websites are accessible

d. Volumes: Mount (bind) a directory from the host file system to Nginx containers and update the contents of index.html in the host file system, re-deploy and confirm in the browser that the web page's content is updated.

Solution:

docker-compose (Multi-build):

Creating web directories for host bind-mount:

Two host-side directories named *webA* and *webB* have been created so that the container's */usr/share/nginx/html/* directory can be replaced with the host folder using bind-mount. This allows web page content to be controlled from the host-side *webA/webB* folders without having to rebuild the container.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ mkdir -p containerB
```

Copying index.html of ContainerA/B to webA/webB:

Separate index.html files of Container A and Container B are copied to host-side *webA* and *webB*. When bind mount is done, the HTML folder inside the container is replaced with the host folder, so the host-side file is served as the final content.

Copying index.html of ContainerA to webA/webB:

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ cp containerA/index.html webA/index.html
```

Copying *containerB*'s index.html to *webA/webB*:

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ cp containerB/index.html webB/index.html
```

docker-compose.yml created (Multi-build + ports + volumes):

The *docker-compose.yml* file has been created so that two services (A and B) can be built+run at the same time.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ nano docker-compose.yml
```

Here, *build: ./containerA* and *build: ./containerB* are used to multi-build from separate Dockerfiles. *Ports:* is used to map the host port and *volumes:* is used to bind-mount the host folder (*webA/webB*) to the web root *(/usr/share/nginx/html)* of the container in read-only *(:ro)* mode.

```
 GNU nano 8.4                                  docker-compose.yml
services:
  a:
    build: ./containerA
    container_name: nginx-a-st24
    ports:
      - "8080:80"
    volumes:
      - ./webA:/usr/share/nginx/html:ro

  b:
    build: ./containerB
    container_name: nginx-b-st24
    ports:
      - "9090:80"
    volumes:
      - ./webB:/usr/share/nginx/html:ro

                                [ Read 16 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo    M-A Set Mark    M-] To Bracket  M-B Previous
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo    M-6 Copy        ^B Where Was    M-F Next
```

Multi-build:

Using this command, two services are started simultaneously in build and detached mode (-d). By giving *--build*, it is easy to change the *Dockerfile* and automatically build it if a new build is needed.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ docker compose up -d --build
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 0.2s (14/14) FINISHED
 => [b internal] load build definition from Dockerfile
 => => transferring dockerfile: 104B
 => [a internal] load build definition from Dockerfile
 => => transferring dockerfile: 105B
 => [b internal] load metadata for docker.io/library/nginx:1.23.3
 => [a internal] load .dockerignore
 => => transferring context: 2B
 => [b internal] load .dockerignore
 => => transferring context: 2B
 => [a internal] load build context
 => => transferring context: 912B
 => [b 1/2] FROM docker.io/library/nginx:1.23.3
 => [b internal] load build context
 => => transferring context: 1.14kB
 => CACHED [b 2/2] COPY index.html /usr/share/nginx/html/index.html
 => CACHED [a 2/2] COPY index.html /usr/share/nginx/html/index.html
 => [a] exporting to image
 => => exporting layers
 => => writing image sha256:d02644d6b596e7c357d9745ff1d35cb38648f790bf3e81abc98e483339170618
 => => naming to docker.io/library/task2-a
 => [b] exporting to image
 => => exporting layers
 => => writing image sha256:6504971bd1eed7d22668dd346170d07afe5fab010653bca4bd36c593480eec42
 => => naming to docker.io/library/task2-b
 => [b] resolving provenance for metadata file
 => [a] resolving provenance for metadata file
[+] Running 4/4
 ✔ a                      Built
 ✔ b                      Built
 ✔ Container nginx-a-st24  Running
 ✔ Container nginx-b-st24  Running
```

The output shows that both a and b have been built and at the end we see:

- Container *nginx-a-st24* Running
- Container *nginx-b-st24* Running

This means that both containers have been successfully started.

Checking:

The following command shows the running compose service/containers, status and port mapping. This indicates that the *8080/9090* mapping is correct.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ docker compose ps
NAME          IMAGE     COMMAND                  SERVICE   CREATED         STATUS          PORTS
nginx-a-st24  task2-a   "/docker-entrypoint.…"   a         51 minutes ago  Up 46 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
nginx-b-st24  task2-b   "/docker-entrypoint.…"   b         51 minutes ago  Up 51 minutes   0.0.0.0:9090->80/tcp, [::]:9090->80/tcp
```

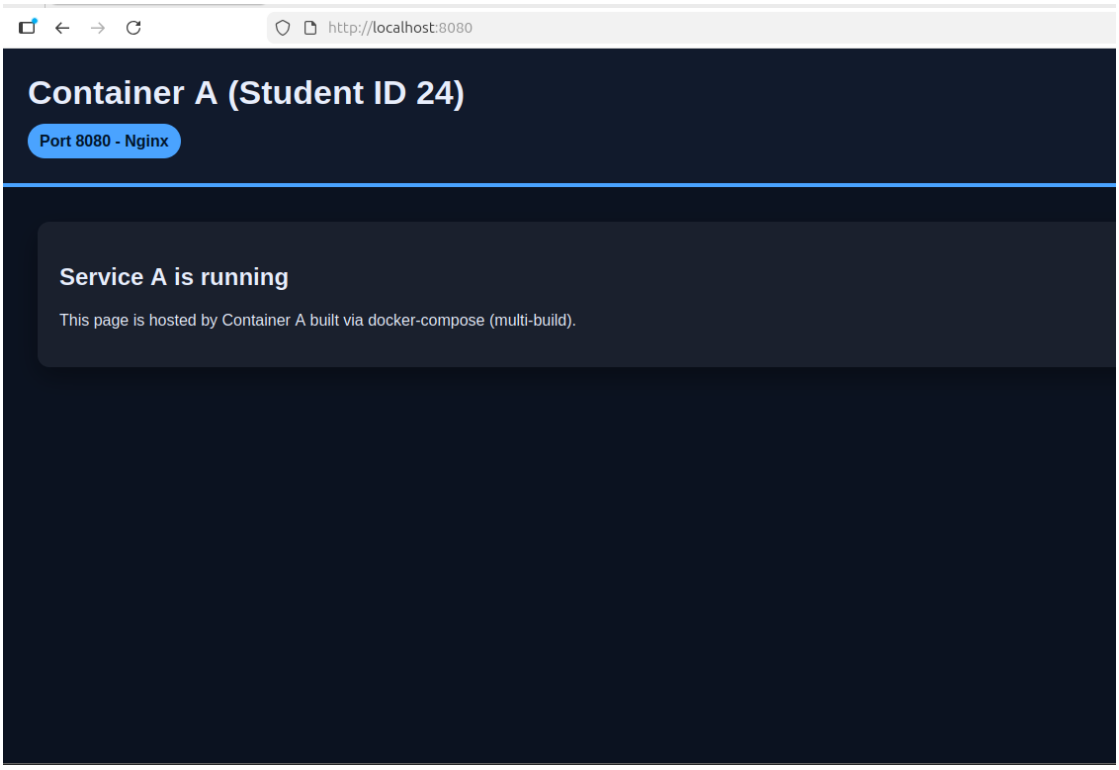
Host port *8080* is for Container A, and Host port *9090* is for Container B.

Both IPv4 and IPv6 bindings are done.

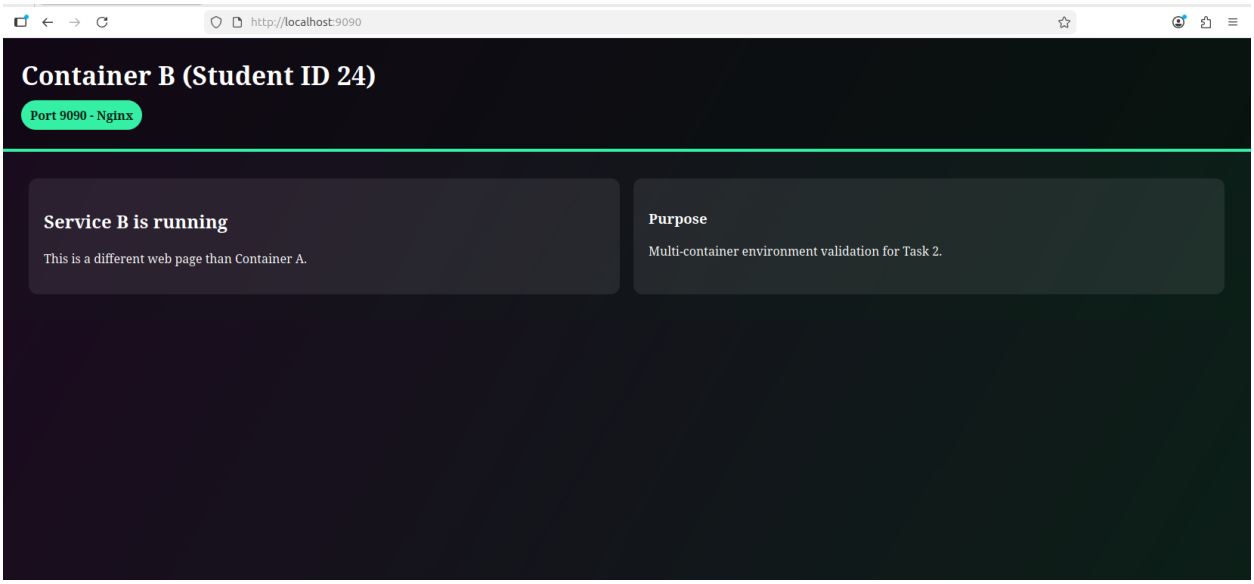
Confirm both websites are accessible:

Open the browser and open the following two URLs separately, where-

- ***http://localhost:8080*** **(Container A page)**



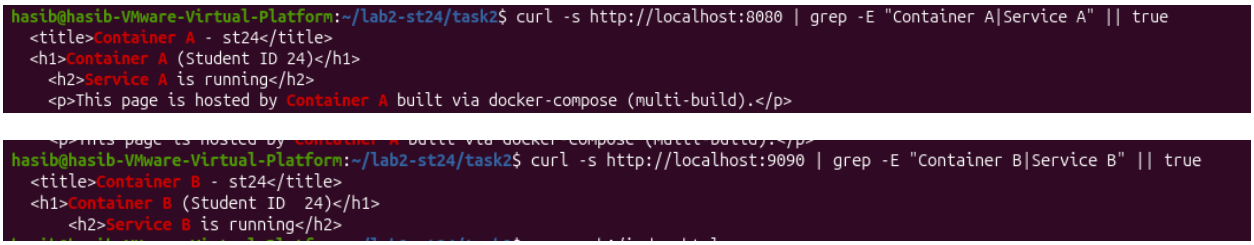
- ***http://localhost:9090*** **(Container B page)**



Two URLs showed different web pages where two containers served two different contents, and port mapping worked properly. This verified that the websites of both Container A and Container B are accessible.

CLI

The *HTTP* response was viewed using *curl*, and specific text matches were performed using grep to confirm that the page from the correct container was loaded.



The output shows that the lines related to Container A / Service A came from *8080* and the lines related to Container B / Service B came from *9090*. This confirms that the website is properly accessible on Container A at *8080* and Container B at *9090.*

Volumes test:

Here, bind-mount is verified. The compose file has ./webA:/usr/share/nginx/html:ro, so the web root (/usr/share/nginx/html) inside the container is actually served from the host-side webA folder. Therefore, editing the host-side webA/index.html file changes the page content of the container, which is proof of the volumes test.

Host-side update:



Then the following new line is added below the line “Service A is running”:



In this, the host-side file has been successfully updated, and the bind-mount-based volume update has been prepared by changing the content.

Re-deploy or restart A:

The re-deploy condition has been met by restarting Container A with the following command, and the new host-side content has been applied.



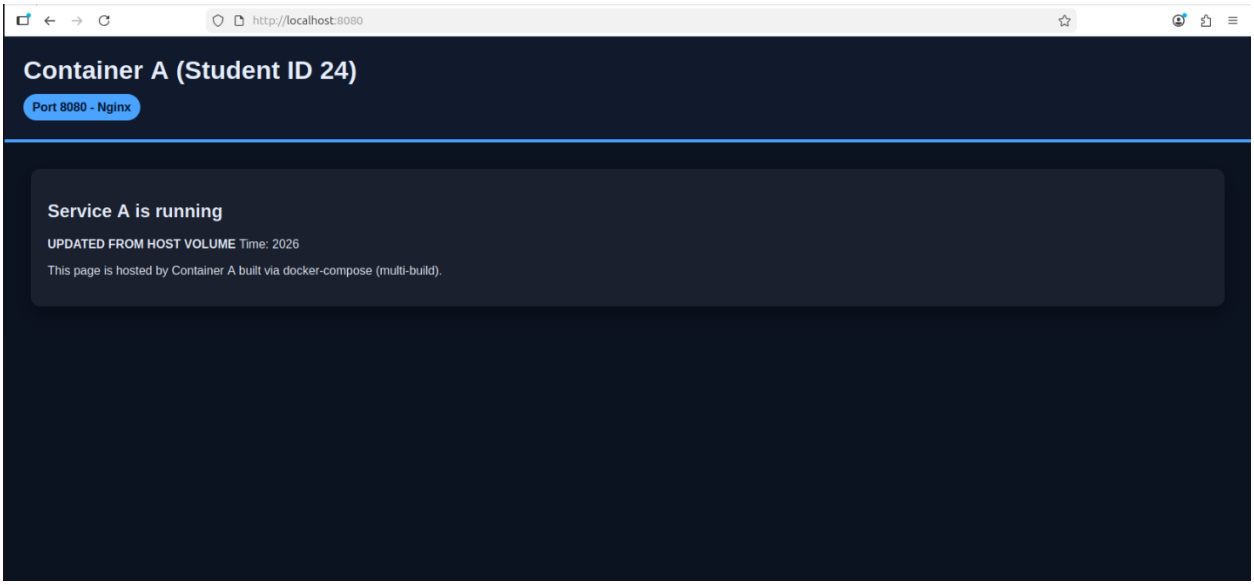
The output shows that the nginx-a-st24 container has been successfully restarted.

Browser refresh:

Open the URL of Container A in the browser and refresh:

- ***http://localhost:8080***

After refreshing, the newly added line "*UPDATED FROM HOST VOLUME … Time: 2026*" is visible on the web page, which proves that the updated file has been served from the bind-mount volume. The web page of Container A has changed according to the host volume update.



Similarly, update Container B:

The same volume test has been done on Container B as well. Because, *./webB:/usr/share/nginx/html:ro* bind-mount is given in compose. So, editing host-side *webB/index.html* updates Container B page.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ nano webB/index.html
```

Here, a new line has been added:

```
<div class="box">
  <h2>Service B is running</h2>
  <p><b>UPDATED FROM HOST VOLUME </b> Time: 2026</p>
```

Then the B service has been restarted:

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ docker compose restart b
[+] Restarting 1/1
 ✔ Container nginx-b-st24  Started
```

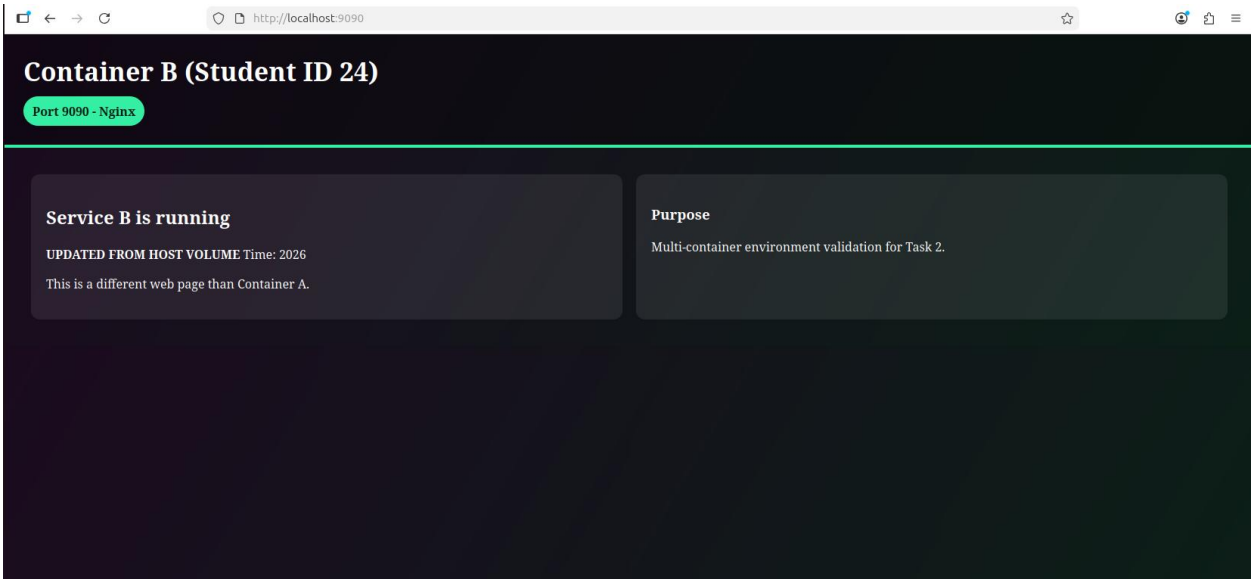
The above output confirms that Container B has been restarted successfully.

Checking with Browser:

Opening the URL of Container B in the browser and refreshing:

- ***http://localhost:9090***

After refresh, "*UPDATED FROM HOST VOLUME … Time: 2026*" is also visible on the Container B page, indicating that the volume bind has worked properly.



From the above result, it can be understood that Container B is also showing changed content according to the host volume update.

3. Configure L7 Loadbalaner a. Install Nginx in the host machine or add a third container in the docker-compose that will act as loadbalancer, and configure it in front of two containers in a manner that it should distribute the load in a Weighted Round Robin approach.

b. Access the page of Nginx ALB and validate, it is load-balancing the traffic (you see two different content per page reload).

Solution:

L7 Load Balancer (Weighted Round Robin)

Here we have added a 3rd container, which distributes traffic from the front.

Creating a Load balancer config folder:

A separate folder named *loadbalancer* has been created to keep the Nginx configuration of the load balancer to manage *Nginx.conf* cleanly.

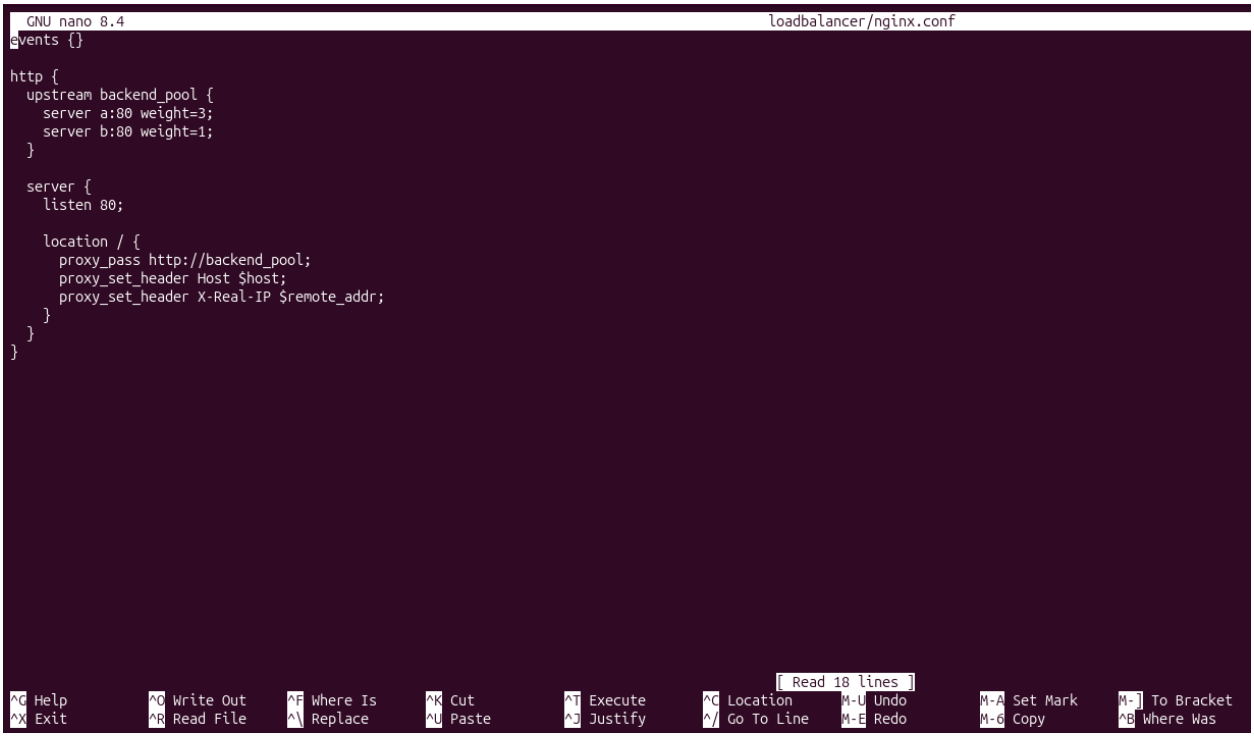
```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ mkdir -p loadbalancer
```

Adding Nginx config (Weighted Round Robin):

Here, the *loadbalancer/nginx.conf* file has been created using the following command.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ nano loadbalancer/nginx.conf
```

Weighted Round Robin is set up inside upstream *backend_pool* with *server a:80 weight=3*; and *server b:80 weight=1;*. A backend gets approximately 3% of the traffic, and the B backend gets 1%. Then all incoming requests are sent to that upstream pool using *proxy_pass*.



Adding LB part to docker-compose.yml:

A third service named *lb* has been added to the *services: of docker-compose.yml.* It will run as a load balancer using the *nginx:1.23.3* image. Port mapping *7070:80* has been done to access *LB* from the host machine. In addition, the *nginx*. The conf file has been bind-mounted and placed in */etc/nginx/nginx.conf*, and it has been ensured with *depends_on* that a and b are ready first.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ nano docker-compose.yml
```

Then the *lb:* service was added to the end of the previous file. A third container, *nginx-lb-st24*, was added to Compose, which acted as a front-end L7 load balancer.

```
 GNU nano 8.4                                                    docker-compose.yml *
services:
  a:
    build: ./containerA
    container_name: nginx-a-st24
    ports:
      - "8080:80"
    volumes:
      - ./webA:/usr/share/nginx/html:ro

  b:
    build: ./containerB
    container_name: nginx-b-st24
    ports:
      - "9090:80"
    volumes:
      - ./webB:/usr/share/nginx/html:ro

  lb:
    image: nginx:1.23.3
    container_name: nginx-lb-st24
    ports:
      - "7070:80"
    volumes:
      - ./loadbalancer/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - a
      - b

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket   M-B Previous
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^B Where Was     M-F Next
```

After saving the Compose file, there are 3 services: a, b, and lb in the final setup.

Rebuild and Deploy:

All services (A, B, LB) have been built and run in detached mode using the following command. Because of *--build, the A/B* image has been rebuilt.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ docker compose up -d --build
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 0.2s (14/14) FINISHED
 => [b internal] load build definition from Dockerfile
 => => transferring dockerfile: 104B
 => [a internal] load build definition from Dockerfile
 => => transferring dockerfile: 105B
 => [a internal] load metadata for docker.io/library/nginx:1.23.3
 => [a internal] load .dockerignore
 => => transferring context: 2B
 => [b internal] load .dockerignore
 => => transferring context: 2B
 => [a internal] load build context
 => => transferring context: 32B
 => [b 1/2] FROM docker.io/library/nginx:1.23.3
 => [b internal] load build context
 => => transferring context: 32B
 => CACHED [a 2/2] COPY index.html /usr/share/nginx/html/index.html
 => [a] exporting to image
 => => exporting layers
 => => writing image sha256:aa3f5ecf24954cc5d48101e1029bf46e076bdd49ad6ee84a29c9819bbda02410
 => => naming to docker.io/library/task2-a
 => CACHED [b 2/2] COPY index.html /usr/share/nginx/html/index.html
 => [b] exporting to image
 => => exporting layers
 => => writing image sha256:6895f737e81f216858430affe42b554241ca6f2f881e45bc19bce5afcf4afd4e
 => => naming to docker.io/library/task2-b
 => [a] resolving provenance for metadata file
 => [b] resolving provenance for metadata file
[+] Running 5/5
 ✔ a                       Built
 ✔ b                       Built
 ✔ Container nginx-b-st24  Running
 ✔ Container nginx-a-st24  Running
 ✔ Container nginx-lb-st24 Started
```

The output shows that the entire stack including three containers A, B and LB has been successfully started.

Checking:

The following command shows the status and port mapping of all running services.

```
hasib@hasib-VMware-Virtual-Platform:~/lab2-st24/task2$ docker compose ps
NAME            IMAGE          COMMAND                  SERVICE   CREATED          STATUS          PORTS
nginx-a-st24    task2-a        "/docker-entrypoint.…"   a         8 hours ago      Up 13 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
nginx-b-st24    task2-b        "/docker-entrypoint.…"   b         8 hours ago      Up 11 minutes   0.0.0.0:9090->80/tcp, [::]:9090->80/tcp
nginx-lb-st24   nginx:1.23.3   "/docker-entrypoint.…"   lb        31 seconds ago   Up 30 seconds   0.0.0.0:7070->80/tcp, [::]:7070->80/tcp
```

From the output, it can be seen that Container A is accessible from host *8080* and Container B is accessible from host *9090.* Load Balancer is accessible from host *7070* and distributes traffic from the front.

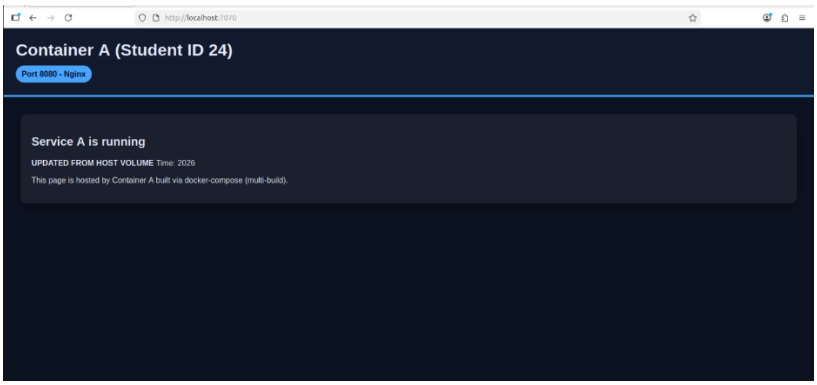
Accessing ALB and Validating load balancing:

Load balancer URL is opened in browser:

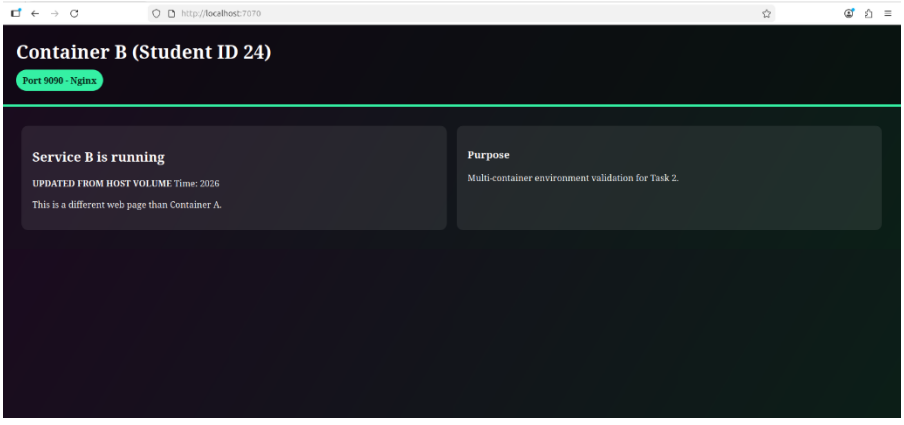
- ***http://localhost:7070***

When you refresh (F5) repeatedly, sometimes A and sometimes B pages are served from upstream pool. Because it is weighted, A content is seen more often (ratio 3:1).

- ***Container A content***



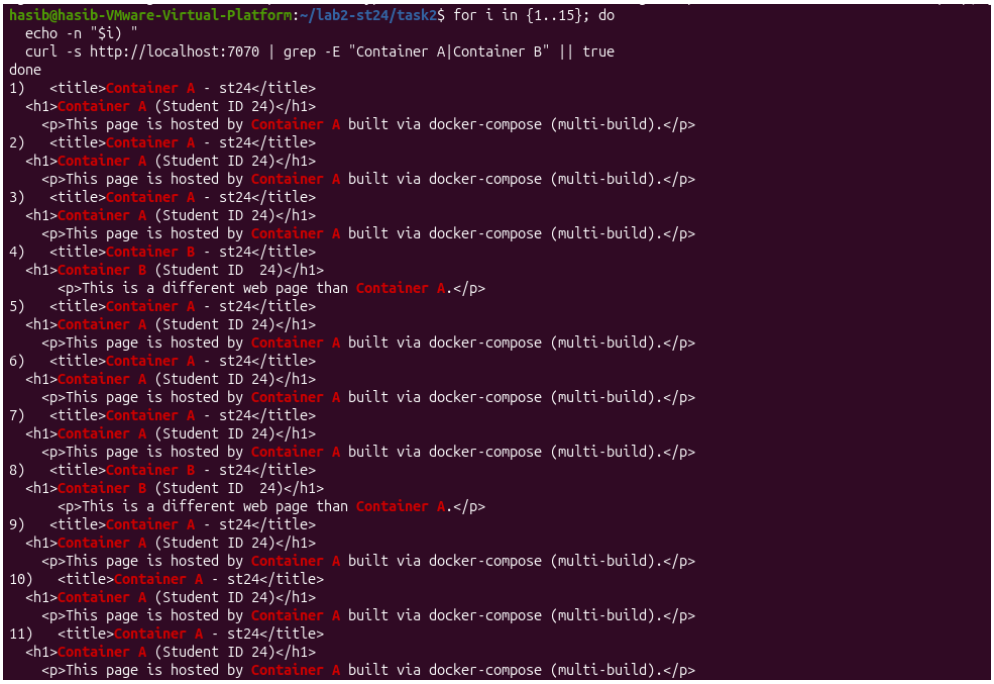
- ***Container B content***

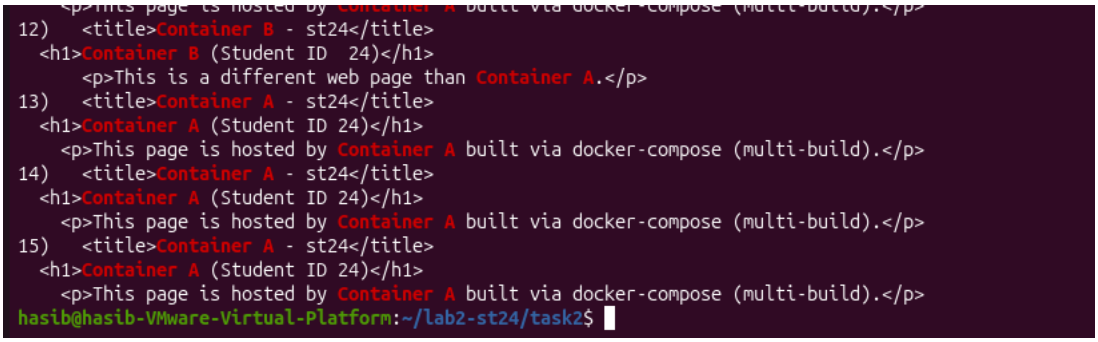


When refreshing, sometimes Container A content comes, sometimes Container B content comes, which is load-balancing proof. A comes more often, Weighted RR proof.

Tested 15 times with CLI:

This loop hit the LB endpoint 15 times and extracted Container A and Container B matching lines from the response. This proves that different backend responses came from the same endpoint.





CLI test confirmed that when a request is made from LB *endpoint 7070*, sometimes A's and sometimes B's response is received, and A is coming more often, which means that Weighted RR is successful.

References

[1] Docker Inc., "*docker image pull*". Available: https://docs.docker.com/reference/cli/docker/image/pull/

[2] Canonical Ltd., "*docker-image-pull(1) - Pull an image or a repository from a registry*". Available: https://manpages.ubuntu.com/manpages/bionic/man1/docker-image-pull.1.html

[3] Docker Inc., "*docker container run*". Available: https://docs.docker.com/reference/cli/docker/container/run/

[4] Docker Inc., "*Running containers*". Available: https://docs.docker.com/engine/containers/run/

[5] Docker Inc., "*docker image (subcommands incl. image ls)*". Available: https://docs.docker.com/reference/cli/docker/image/

[6] Docker Inc., "*docker container ls (docker ps)*". Available: https://docs.docker.com/reference/cli/docker/container/ls/

[7] Docker Inc., *"Linux post-installation steps for Docker Engine"*. Available: https://docs.docker.com/engine/install/linux-postinstall/

[8] Docker Inc., "*Compose file reference*". Available: https://docs.docker.com/reference/compose-file/

[9] Docker Inc., "*Compose Build Specification (build)*". Available: https://docs.docker.com/reference/compose-file/build/

[10] Docker Inc., *"Bind mounts*". Available: https://docs.docker.com/engine/storage/bind-mounts/

[11] Docker Inc., *"docker compose up"*. Available: https://docs.docker.com/reference/cli/docker/compose/up/

[12] F5 NGINX, *"HTTP Load Balancing"*. Available: https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/

[13] NGINX, Inc., *"Using nginx as HTTP load balancer"*. Available: https://nginx.org/en/docs/http/load_balancing.html

[14] NGINX, Inc., *"ngx_http_upstream_module"*. Available: https://nginx.org/en/docs/http/ngx_http_upstream_module.html

[15] NGINX, Inc., *"ngx_http_proxy_module (proxy_pass)"*. Available: https://nginx.org/en/docs/http/ngx_http_proxy_module.html